

AI OPERATIONS PORTFOLIO · USER MANUAL

Autonomous Planning System

Scheduled cloud agents that plan your week and verify what you actually did.

John White

AI Operations & Optimization

[linkedin.com/in/johnwhiteimse](https://www.linkedin.com/in/johnwhiteimse) · github.com/johnwhite-ai/ai-operations-portfolio

Contents

- 01** Executive Summary
- 02** The Problem
- 03** Architecture
- 04** The Differentiator: It Verifies Reality, Not Memory
- 05** How to Use It
- 06** A Worked Example
- 07** Design Rationale
- 08** What This Project Demonstrates

01 Executive Summary

The Autonomous Planning System is two scheduled agents that run in the cloud on a cron. One builds a weekly plan every Friday; one builds a daily review every weekday morning. Each reads real context across five tools, drafts a proposal where every action is a checkbox, and waits for one-click human approval before anything is created.

Its standout feature is that it verifies what you actually did yesterday against the evidence — commits, sent messages, file edits — rather than trusting your memory. Total human time is five to ten minutes a day.

THE ONE-LINE VERSION

Planning tools make you do the planning, and none can tell you whether yesterday's commitments actually happened. This one does the synthesis and checks reality against the evidence — then waits for your approval before acting.

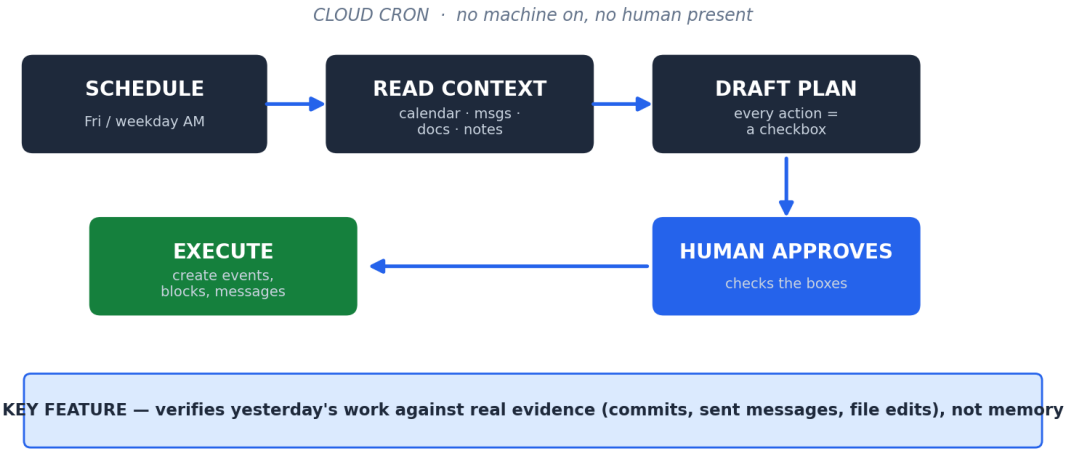
02 The Problem

Planning tools hand you an empty calendar and a task list and leave the synthesis — what matters this week, what slipped yesterday, what to move — entirely to you. And almost none can answer the one question that actually matters at the start of a day: did the things I committed to yesterday actually happen?

Relying on memory for that is unreliable, and quietly dropped commitments are how trust erodes. The hard part of planning is not making the list; it is reconciling intent against reality, every day, across every tool where the work actually lands.

03 Architecture

Two agents on a schedule, with a human-approval gate so the system proposes but never acts on its own. Both run in the cloud — no machine needs to be on, and no human needs to be present.



Scheduled agents read context, draft a plan where every action is a checkbox, the human approves, and only then do actions execute.

The three building blocks

Component	Role
Weekly Planner	Friday agent: reconciles prep folders, builds next week's plan in a fixed structure, drafts a prep note per task block.
Daily Reviewer	Weekday agent: verifies yesterday's work against evidence, then proposes today's adjustments.
Approval Skill	The human-in-the-loop step: scans the proposal for checked items, creates them via the tools, logs an audit trail.

Context as the control surface

A single priorities/context document steers every run — current constraints, active projects, and a numbered set of planning principles the agents cite by number. When a run drifts from reality, the fix is to add the missing context to that one document, not to edit the agents. The maintenance loop is literally: "what context would have prevented this?"

04 The Differentiator: It Verifies Reality, Not Memory

The highest-value feature is the yesterday recap with evidence. For each task block from the day before, the agent does not ask whether it was done — it checks the evidence across the connected tools and assigns a status.

Status	Evidence required
Verified Complete	A concrete artifact: a commit, a sent message, or a file modified at a specific time by the user
Likely Complete	Partial or indirect evidence
Unverified	No trace found — flagged for the human to finish today or formally drop

It also checks commitments made yesterday ("I'll send X by EOD" — did it ship?), inbound asks (did the human reply?), and meetings attended.

WHY THIS IS THE KILLER FEATURE

Knowing "the document was edited at 4:47 PM yesterday and the system confirms it was you" is a far stronger signal than relying on memory. It catches dropped commitments before they become problems — and it is what makes the system trustworthy rather than just another planner.

05 How to Use It

One-time setup

1. Connect the tool integrations (calendar, messaging, docs, notes, repo).
2. Write the context/priorities document – constraints, projects, principles.
3. Schedule the two agents on cron (Weekly: Friday AM; Daily: weekday AM).
4. (Optional) Add a session-start hook to auto-pull the latest plan files.

The daily rhythm (5–10 minutes)

Morning: a notification says the daily review is ready.

- > Read the Yesterday Recap: caught up? Anything Unverified to finish/drop?
- > Read the Approval section: what will run if you approve.
- > Check the boxes you want, say "approve daily changes."
- > The approval skill creates the events/blocks/messages and logs it. ~30s.

Friday: same flow against the weekly plan ("approve weekly plan").

06 A Worked Example

A weekday morning. The Daily Reviewer has already run in the cloud.

- It reads yesterday's three task blocks and verifies each: "Draft proposal" — Verified Complete (file modified 4:47 PM); "Send follow-up" — Verified Complete (message sent 2:11 PM); "Review analysis model" — Unverified (no artifact found).
- It checks commitments: "will send the summary by EOD" — confirmed sent 5:52 PM. An inbound timeline ask — not yet answered, queued for this morning.
- It proposes today: a 9:00 block to answer the timeline ask, the analysis-model block moved earlier to protect it before it slips again, deep-work block unchanged.
- The Approval section lists three checkboxes. The human checks two, says "approve daily changes," and the skill creates them and logs it.

THE POINT

Nothing reached the real calendar without a human check, and the system surfaced a slipping commitment the human might have forgotten. That is the whole purpose.

07 Design Rationale

Why a human-approval gate instead of full autonomy?

An agent that silently creates meetings and sends messages is one wrong inference away from an embarrassing mistake. Splitting "agent proposes" from "human approves" keeps the heavy synthesis automated while keeping judgment and control with the person — and the audit log makes every action traceable after the fact. That is what makes an autonomous system safe to actually run.

Why verification over self-report?

Self-report is the weakest signal in planning — it is exactly what fails when you are busy. Checking real artifacts turns the daily review from a to-do list into an accountability system, which is where the real value is.

08

What This Project Demonstrates

Capability	How this project proves it
Scheduled autonomous operation	Cloud cron agents that run with no machine on and no human present
Multi-source reconciliation	Synthesizes calendar, messages, docs, and notes into one plan
Evidence-based verification	Confirms completed work against artifacts, not self-report
Human-in-the-loop design	Proposes everything, executes only what is approved, full audit trail
Context as the control surface	A single priorities doc steers every run; drift is fixed by editing context, not code